Systematic Model Selection & Cross-Validation

A Colorful Guide to Diagnosing, Splitting, and Selecting Machine Learning Models



Tutorial Report | Train / Cross-Validation / Test Methodology

Covers: diagnostics, the three-way split, bias vs. variance, polynomial degree selection, and neural network architecture selection.

Section color key: Full Walkthrough (coral) · Core Concept (teal) · Why It Matters (amber) · The Mechanism (blue) · Worked Examples (purple) · Foundational vs. Incidental (orange) · Real-World Relevance (green) · Common Misconceptions (coral) · FAQs (teal)

1. Full Walkthrough

In the lifecycle of a machine learning project, the primary differentiator between an effective team and an ineffective one is decision-making speed. Some teams spend six months building a system that a more skilled group could finish in a fortnight — and the gap rarely comes from typing speed. It comes from knowing **what to do next** when a model underperforms.

Picture a regularised linear regression model for housing prices whose prediction errors are unacceptably high. There is a daunting list of possible paths forward:

- Spend months acquiring more training examples.
- Prune the feature set to prevent overfitting.
- Engineer additional features to capture more nuance.
- Add polynomial features (x_1^2, x_2^2, x_1x_2).
- Fine-tune the regularisation parameter (λ) up or down.

Without a systematic approach, picking a path is guesswork. This is where **diagnostics** come in — targeted tests that reveal how an algorithm is actually failing, so effort goes where it will actually help.

Historically, evaluation relied on visual plotting: with a single feature like house size, a “wiggly” curve makes overfitting obvious to the eye. But real models typically use many features (size, bedrooms, floors, age, ...) and a four-dimensional function cannot be plotted. The strategy has to evolve — from visual inspection, to a Train / Test split, to the more robust Train / Cross-Validation / Test split shown below.



The three data subsets and the role each one plays.

*Aside: the "Cross-Validation set" is also called the "Validation set" or the "Development set" — most practitioners just say **"dev set"** because it’s faster to say.*

Takeaway: Diagnostics replace guesswork with evidence. Because we can no longer eyeball performance in high dimensions, we need a structured way — the three-way data split — to see how a model is really doing.

2. Core Concept

The central pillar of this methodology is **unbiased evaluation**. To understand how a model will perform in the real world — its *generalisation error* — it must be tested on data that never influenced its parameters or its structural design.

Think of a student preparing for a final qualification. The **Training Set** is their textbook and daily exercises. The **Cross-Validation Set** is a series of mock exams the teacher uses to decide which textbook or teaching style works best. The **Test Set** is the final, official exam. If the student or teacher sees the final exam questions while choosing the textbook, the resulting score is an “over-optimistic” reflection of real knowledge.

Takeaway: An evaluation is only meaningful if the data being scored on was kept completely separate from every decision that shaped the model.

3. Why It Matters

Engineering efficiency is the art of avoiding “dead-end” development.

- **The High-Dimensional Blind Spot** — intuition works in 2D, but visual plots are useless across many features (x_1 through x_4). Statistical rigor is needed to detect failure where the eye cannot.
- **Resource Preservation** — a diagnostic can prove that collecting more data will not help, saving a team months of unnecessary data acquisition.
- **Preventing Over-optimism** — training error is a deceptive metric: a model can reach zero training error by “memorising” the training set while failing to generalise at all.

Takeaway: Skipping diagnostics doesn't just risk a worse model — it risks months of effort spent in the wrong direction entirely.

4. The Mechanism

Phase 1 — The Simple Split

The dataset is partitioned into two subsets, typically using a 70/30 or 80/20 ratio.

- **Training Set** — used to fit the parameters w and b.
- **Test Set** — used to evaluate the resulting model on unseen data.

Phase 2 — Evaluation Metrics (Linear Regression)

To measure performance we calculate the average squared error. Crucially, while the training objective includes a regularisation term (λ) to help fit the model, that term is **excluded** from evaluation metrics.

PYTHON

```python
# Training error (regularisation excluded from reporting)
def j_train(w, b, X_train, y_train):
    m = len(y_train)
    preds = predict(X_train, w, b)
    return sum((preds - y_train) ** 2) / (2 * m)

# Test error uses the exact same formula on held-out data
def j_test(w, b, X_test, y_test):
    m = len(y_test)
    preds = predict(X_test, w, b)
    return sum((preds - y_test) ** 2) / (2 * m)
```

Phase 3 — Classification Metrics (Logistic Regression)

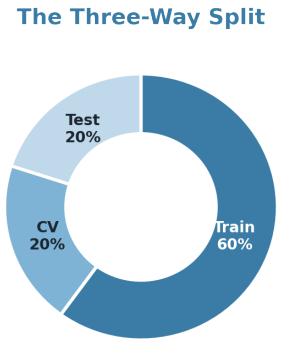
In classification, rather than logistic loss, we often measure the **Misclassification Error** — the fraction of the dataset the algorithm got wrong.

- **Thresholding** — predict $\hat{y} = 1$ if $f_{w,b}(x) \geq 0.5$, otherwise $\hat{y} = 0$.
- **Metric** — J_{cv} and J_{test} are the average number of times $\hat{y}^{(i)} \neq y^{(i)}$.

Phase 4 — The Three-Way Split

To select the best model (e.g. choosing a polynomial degree) without biasing the test set, a 60/20/20 split is used:

- **Training Set (60%)** — fit the parameters w, b.
- **Cross-Validation Set (20%)** — select the model architecture or hyperparameters.
- **Test Set (20%)** — provide the final, unbiased estimate of generalisation error.



A 60/20/20 three-way split.

Formula Breakdown

Variable	Physical Representation in Dataset
`J_train(w,b)`	Average error on the examples used for parameter fitting.
`J_cv(w,b)`	Average error on the subset used for selecting the best model.
`J_test(w,b)`	Average error on the “sealed” subset used for final reporting.
`m_train`	Total number of rows/examples allocated to the training portion.
`m_cv`	Total number of rows/examples allocated to the cross-validation portion.
`m_test`	Total number of rows/examples allocated to the final test portion.

Takeaway: The three-way split isn't just extra bookkeeping: it's what keeps model-selection decisions (like polynomial degree or architecture) from silently leaking into the number you report as “final performance.”

5. Worked Examples

Example 1 — Polynomial Degree Selection

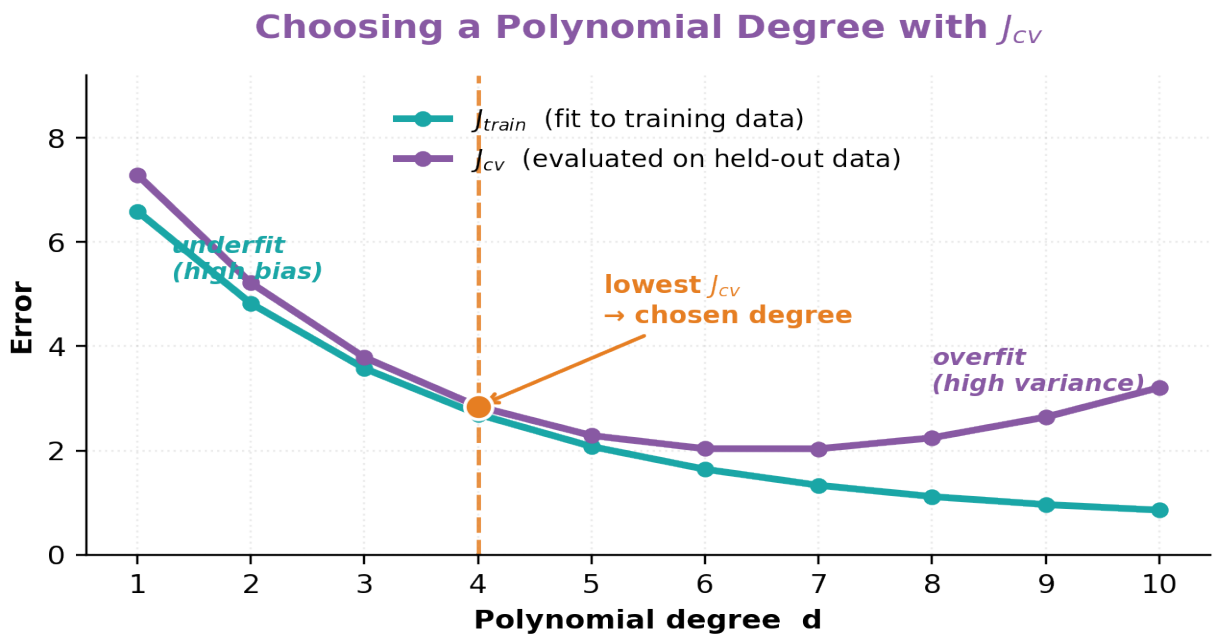
Suppose we are deciding between polynomial degrees d = 1 (linear) through d = 10.

1. Train 10 separate models on the Training Set.
2. Calculate the error for each on the Cross-Validation Set (J_{cv}).
3. Find that d = 4 has the lowest J_{cv} and select it as the final model.
4. Use J_{test} to report how well this d = 4 model actually performs.

PYTHON

```python
# Model selection loop -- degree is chosen using J_cv only
best_degree, best_jcv = None, float('inf')
for d in range(1, 11):
    model = fit_polynomial(X_train, y_train, degree=d)
    error = j_cv(model, X_cv, y_cv)
    if error < best_jcv:
        best_degree, best_jcv = d, error

# Test set is touched exactly once, after selection is final
final_error = j_test(best_degree_model, X_test, y_test)
```



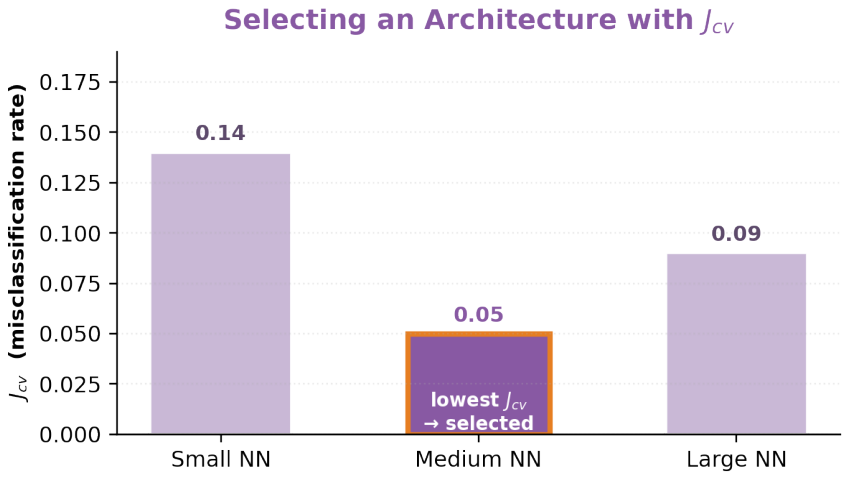
J_train keeps falling as degree increases, but J_cv bottoms out at d = 4 — the point past which the model starts overfitting.

> ***The Flaw:*** *if d = 4 had been picked because it had the lowest Test Error, the Test Set would no longer be a fair indicator. The parameter d would have been "fit" to the test data, producing an optimistic bias.*

Example 2 — Neural Network Architecture

When choosing between a Small, Medium, and Large Neural Network (NN):

1. Fit w and b for all three NNs using the Training Set.
2. Evaluate the fraction of misclassified examples for each on the Cross-Validation Set.
3. Pick the architecture with the lowest J_{cv}.
4. Calculate J_{test} for that specific architecture to estimate real-world performance.



The Medium NN wins on J_cv, so it becomes the architecture reported on J_test.

Takeaway: Both examples follow the same shape: try several candidates, rank them with J_cv, then take one single, final look at J_test. The candidate list can be polynomial degrees, network sizes, feature sets, or λ values — the procedure doesn't change.

6. Foundational vs. Incidental

Foundational (Core Principles)	Incidental (Flexible / Synonymous)
Three-Way Split: the absolute necessity of isolating the selection process from the final evaluation.	Split Ratios: 60/20/20 is a common recommendation, but ratios can vary with total data size.
Regularisation Exclusion: λ is used to train, but error metrics (J_cv, J_test) must only measure prediction accuracy.	Naming Conventions: “Dev set,” “Validation set,” and “Cross-validation set” are used interchangeably.
Generalisation Error: the fundamental goal is measuring performance on unseen examples.	The Name “Cross-Validation”: not considered a great name, but it is the industry standard.

Takeaway: If you remember only one row: the split ratio and the naming can flex, but keeping the selection process isolated from the final evaluation cannot.

7. Real-World Relevance

Without these diagnostics, teams often fall into the **“More Data Trap.”** They observe high error and instinctively spend months and significant budget gathering more data. A simple diagnostic could have revealed that the model was already failing to fit the training data (high bias) — meaning more data would have changed nothing.

This systematic structure is also the engine behind automated **Hyperparameter Optimisation (HPO)**. Modern tools run hundreds of iterations, using J_{cv} to automatically rank and choose settings, so the engineer only ever sees the final, honest performance on the J_{test} set.

Takeaway: The three-way split scales from a single engineer manually picking a polynomial degree all the way up to automated systems running hundreds of hyperparameter trials — the underlying safeguard is identical.

8. Common Misconceptions

Misconception: Training error (J_train) accurately reflects how a model will perform for users.

- **Reality:** J_train is usually much lower than the actual error because the model was specifically optimised to minimise it.

Misconception: The regularisation penalty (λ) should be included in J_test.

- **Reality:** J_test must reflect real-world cost (e.g. how far off the house-price prediction actually was), not the artificial penalty used to constrain weights.

Misconception: It is fine to use the Test Set to choose between two different neural networks.

- **Reality:** If the Test Set influences any choice (architecture, features, or λ), it is no longer an unbiased measure. The Cross-Validation set must be used for these decisions.

Takeaway: Every misconception above traces back to one habit: letting a number that influenced a decision also be the number used to report performance.

9. FAQs

Q: Why is it called “Cross-Validation”?

A: It refers to using an extra dataset to “cross-check” the accuracy of different models. It isn't considered a perfectly descriptive name, but it's what the community settled on.

Q: What is a “Dev set”?

A: Short for “Development set,” it's another term for the cross-validation set — practitioners often prefer it because it's shorter and faster to say.

Q: Why is J_test often higher than J_train?

A: The learning algorithm is specifically designed to minimise J_train. Since the model “sees” the training data while fitting, it naturally performs better on those examples than on unseen test examples.

Q: Can I use the test set to tune my λ or d?

A: No. Doing so creates an “optimistic bias.” You'd be choosing parameters that happen to work well on that specific test set, which may not translate to truly new data in production. All tuning must happen on the Cross-Validation set.

Takeaway: Almost every FAQ answer above reduces to the same rule: the test set gets touched exactly once, after every other decision has already been made.